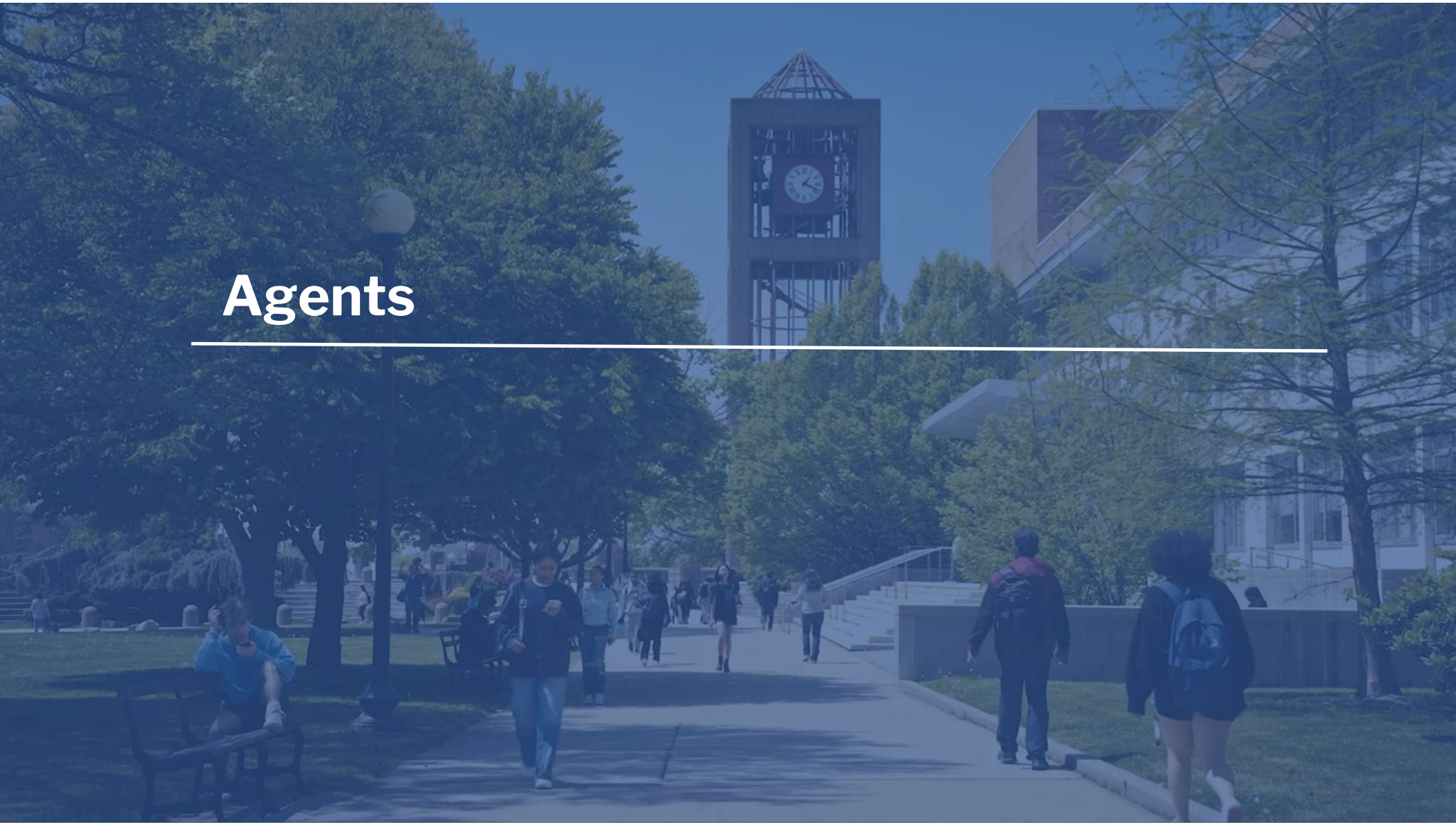
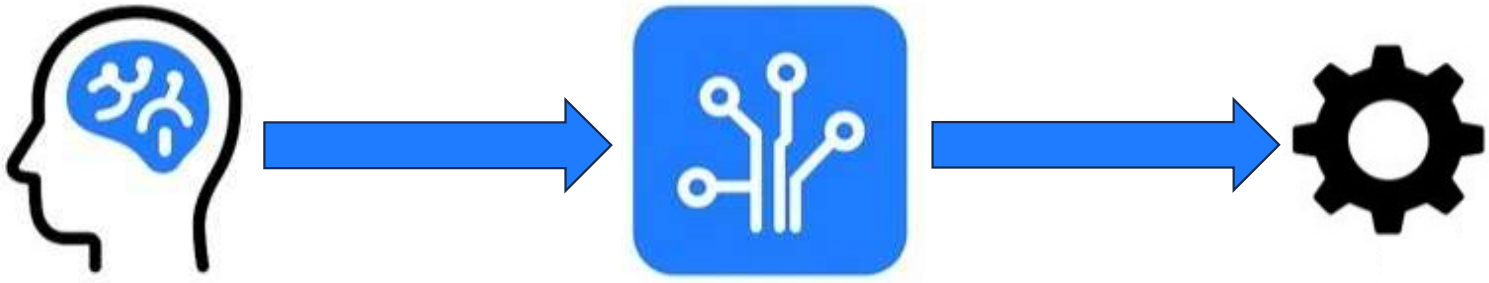

Introduction to Generative AI
(GAI 602)
WEEK 10
CU
NY
SPS


Agents

What are agents



Perceive

The perceive module acts as the agent's sensory interface with the external world. It transforms raw environmental input into structured representations that inform reasoning. This includes handling multimodal data such as text, audio, or sensor signals.

Reason

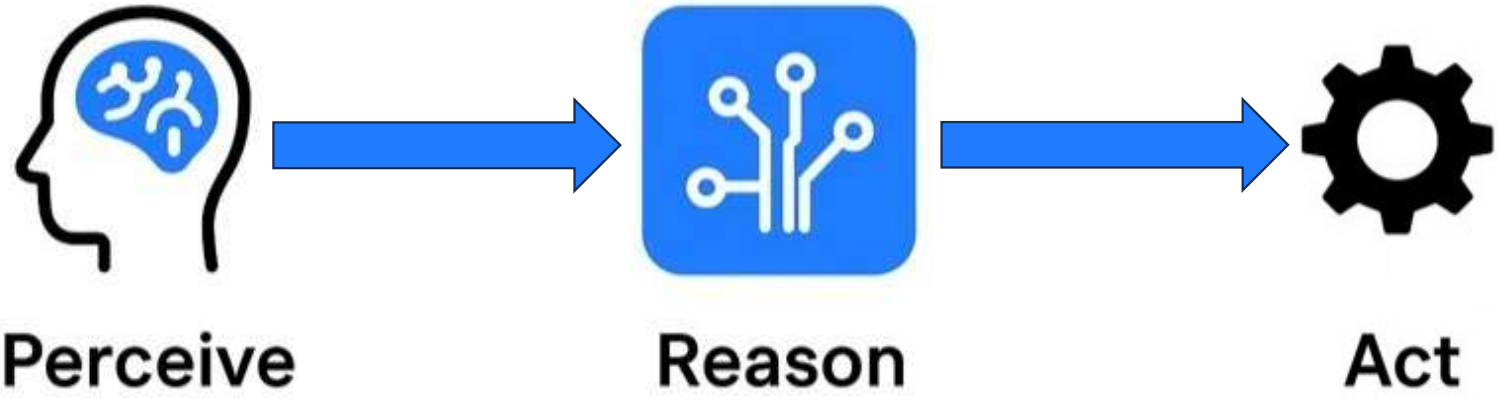
The reason module is the cognitive core of the agent. It evaluates context, formulates intent, and determines appropriate actions. This module orchestrates goal-driven behavior by using both learned knowledge and reasoning.

Act

The act module executes the agent's selected decision by interfacing with either the digital or the physical environment to carry out tasks. This is where intention becomes action. The outputs of the act module feed back into the environment and close the loop.

Source: AWS

What are agents



	Perceive	Reason	Act
Reinforcement Learning:	World Model	Policy	Policy
Generative AI:	LLM	LLM	LLM

Source: AWS



LLM Agents

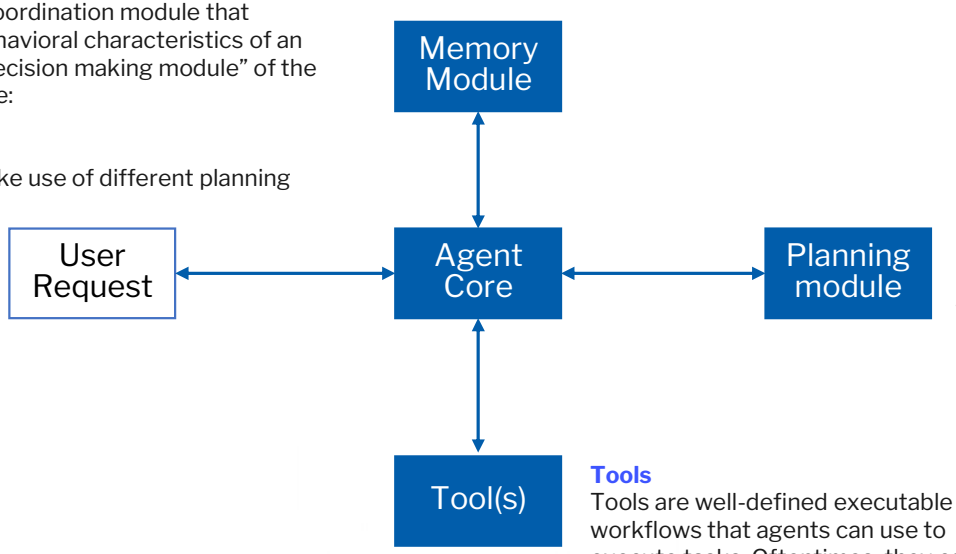
Agentic Problem Solving

Let's take a look at a typical Agent setup.
There are four main modules:

Memory module
Memory modules play a critical role in AI agents. A memory module can essentially be thought of as a store of the agent's internal logs as well as interactions with a user.

Agent core
The agent core is the central coordination module that manages the core logic and behavioral characteristics of an Agent. Think of it as the "key decision making module" of the agent. It is also where we define:

- General goals of the agent
- Tools for execution
- Explanation for how to make use of different planning modules
- Relevant Memory



Planning module
Complex problems, such as analyzing a set of financial reports to answer a layered business question, often require nuanced approaches. With an LLM–powered agent, this complexity can be dealt with by using a combination of two techniques:

- Task and question decomposition
- Reflection or critic

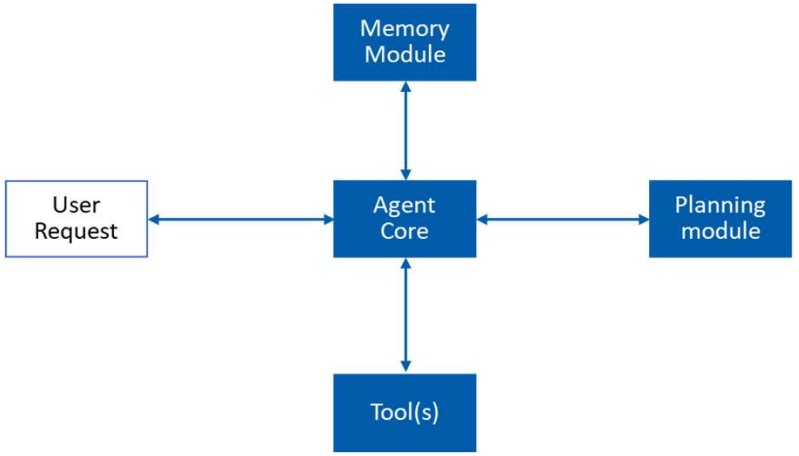
Tools
Tools are well-defined executable workflows that agents can use to execute tasks. Oftentimes, they can be thought of as specialized third-party APIs.

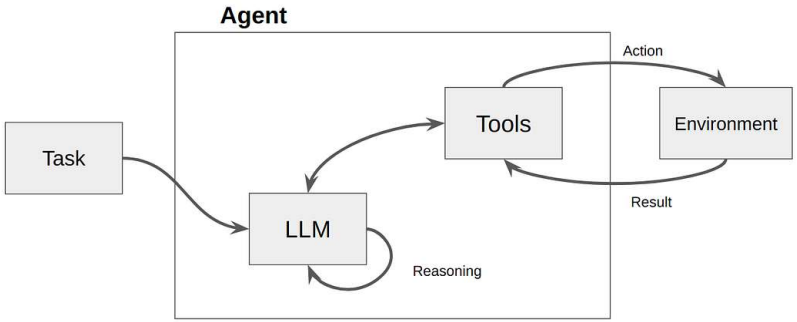
Source: Nvidia

Agentic Problem Solving

How does an Agent work?

1. **Task Understanding**
   Input Analysis: The agent processes user input to understand the task and identifies the goals or sub-goals.
2. **Planning and Reasoning**
   Strategy Development: It breaks down the task into actionable steps, planning the sequence of operations needed to achieve the goals.
3. **Execution**
   Action Implementation: The agent carries out the planned steps, interacting with external systems, APIs, or databases as needed
4. **Interaction and Feedback**
   Continuous Engagement: It interacts with the user or environment for additional information, provides updates, and adjusts actions based on feedback.





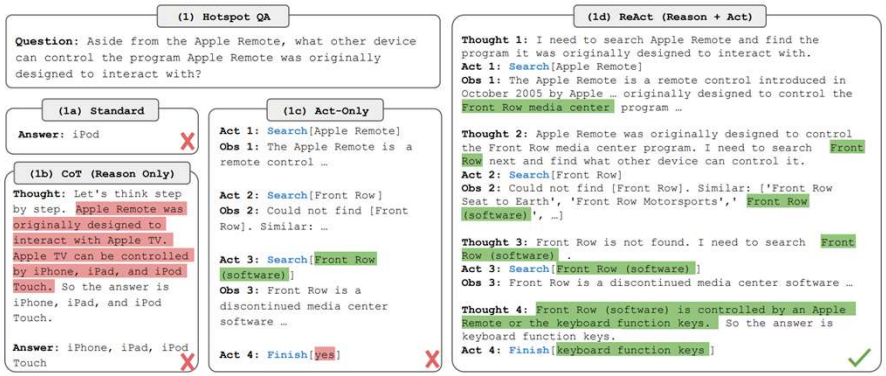
Source: Nvidia

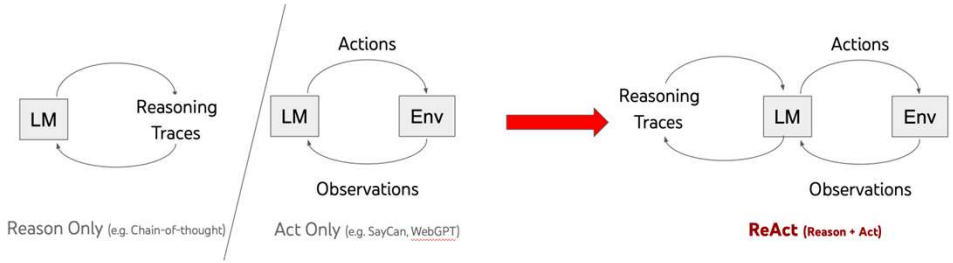
ReAct: Reasoning and Acting

There are a number of ways to define the logical flow of solving tasks for Agents.

We've already seen Chain-of-Thought, but we can also use ReAct

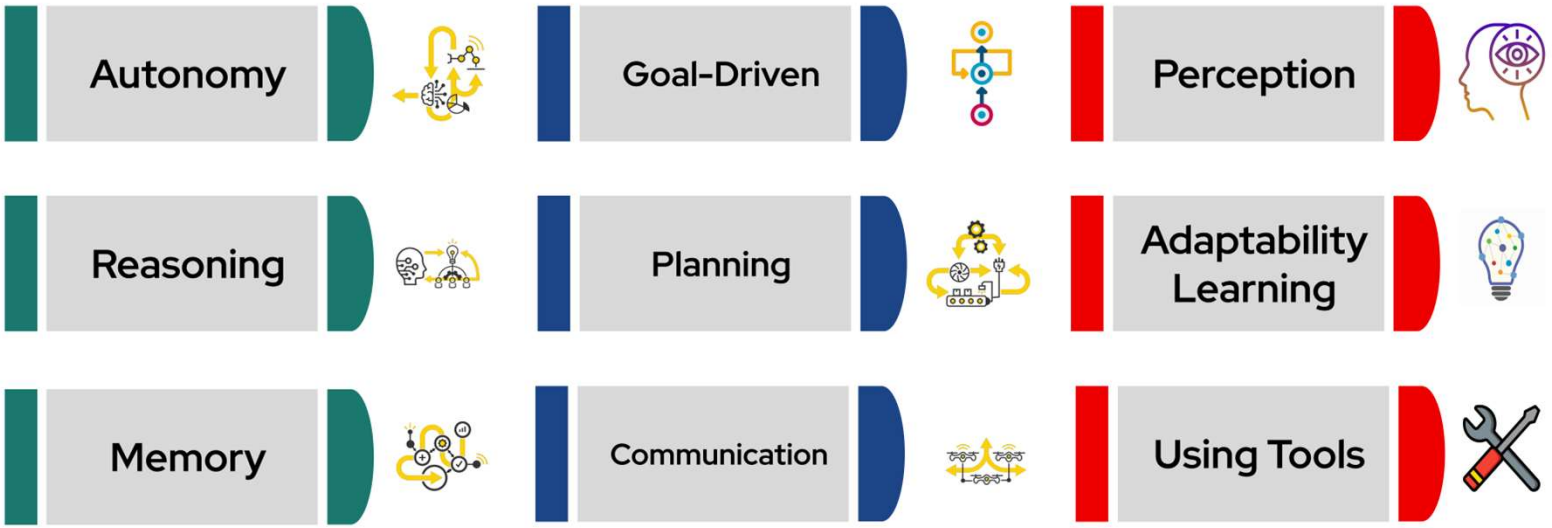
ReAct stands for "Reasoning and Acting" and is a planning approach where the LLM agent dynamically combines reasoning with immediate action in response to the current situation or user input. Unlike other approaches that may involve extensive pre-planning or exploration of multiple possibilities before acting, ReAct focuses on real-time decision-making





Source: Nvidia

Core Features of AI Agents



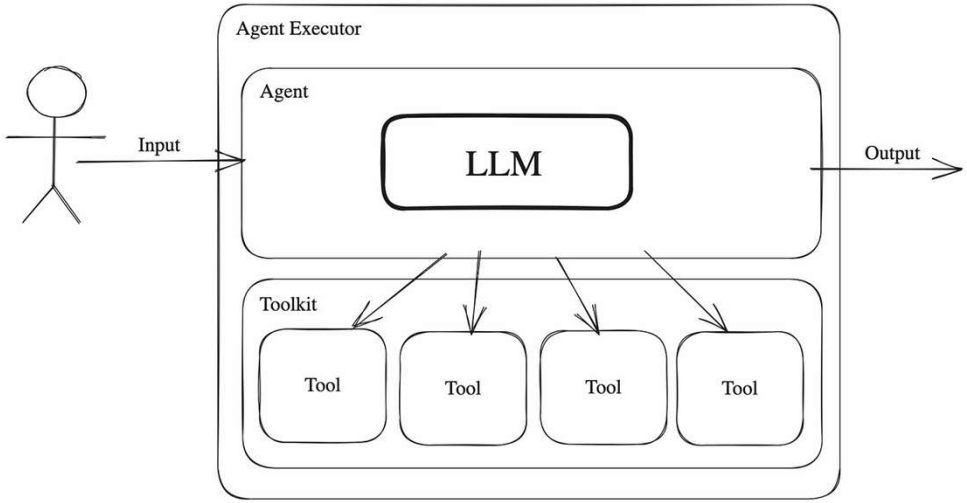
Source: Redhat


Tools & Plugins

Tool Usage

Tool usage and interfacing

- Tools are utilities designed to be called by a model
- Tool inputs are designed to be generated by models
- Tool outputs are designed to be passed back to models
- Tools are needed whenever you want a model to control parts of your code or call out to external APIs



An LLM Agent's tool consists of:

1. The name of the tool
2. A description of what the tool does
3. A JSON schema defining the inputs to the tool
4. A function (and, optionally, an async variant of the function).

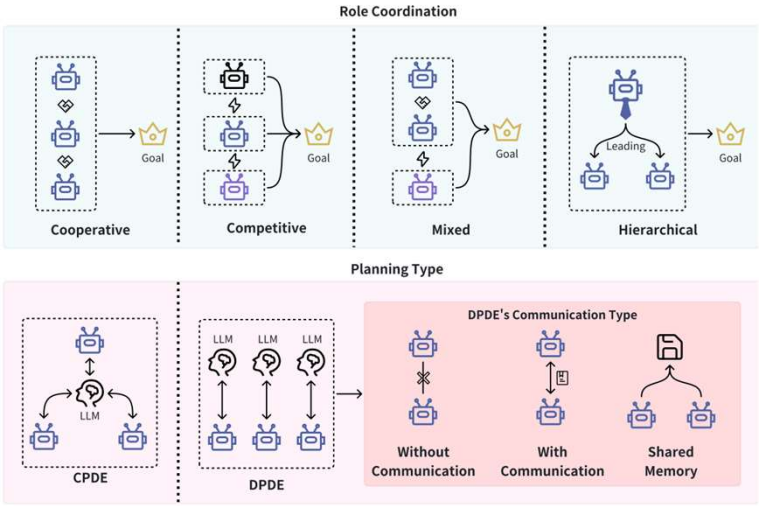
When a tool is bound to a model, the name, description and JSON schema are provided as context to the model.

Given a list of tools and a set of instructions, a model can request to call one or more tools with specific inputs.

Source: Nvidia

Multi-Agent Systems (MAS)

MAS consists of multiple interacting intelligent agents, each with specific domain expertise, working together to achieve complex tasks across multiple domains.



Key Dimensions

- Agent Granularity: From coarse to fine configurations.
- Agent Knowledge: From redundant to specialized expertise.
- Control Distribution: Benevolent/competitive, team/hierarchical.
- Communication Protocols: Blackboard vs. message-based, low-level to high-level content.

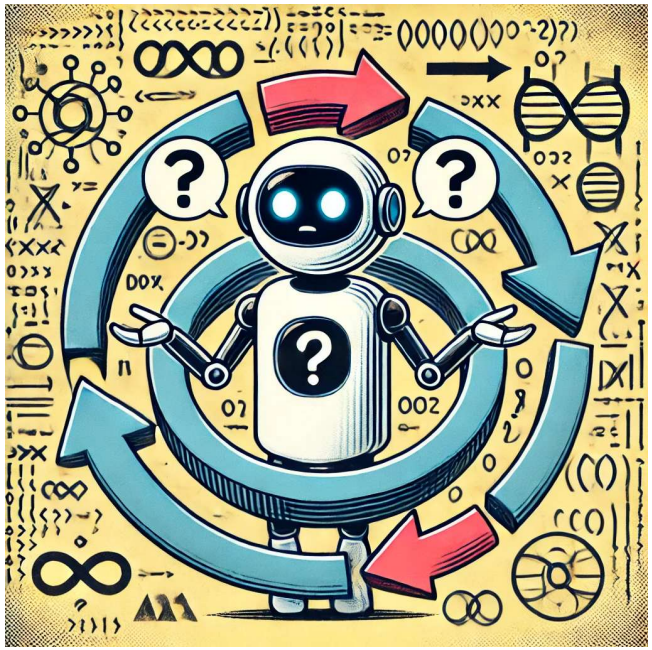
Taxonomy & Classification

- System & Agent Architecture: Focus on function, communication, and agent heterogeneity.
- MAS Archetypes: Homogeneous/heterogeneous, communicating/non-communicating agents.
- Graph Representation: Nodes as agents, edges as communication links.
- Categorization: Focus on multi-role coordination (cooperative, competitive, mixed, hierarchical) and planning types (Centralized/Decentralized).

Source: Nvidia

Common Challenges in Deploying Agents

- **Reliability**
  Ensuring consistent, accurate performance is a major challenge. Reliable agents are crucial for seamless, autonomous operation without human intervention.
- **Entitlement**
  Agents act on a user's behalf, accessing data and performing tasks. Ensuring that agents are only allowed to access data and perform tasks that the user is permitted for is critical to ensure segregation of duties
- **Shadow AI**
  Agents can be easy to setup and run. Ensuring that agents are registered, managed, logged, and can be shutdown in the event of an issue is important for governance
- **Loop Management**
  Agents can get stuck in endless loops, particularly in frameworks like CrewAI. Implementing safeguards to detect and halt these loops is essential to avoid inefficiencies.
- **Tool Customization**
  Tailoring tools to the agent's specific use case improves performance. Custom tools enhance data processing and functionality, making the agent more effective.
- **Self-Checking Mechanisms**
  Incorporating self-checking ensures the quality and relevance of outputs, enabling agents to autonomously detect and correct errors.
- **Explainability**
  Providing clear explanations for decisions and outputs builds transparency and trust, making the agent's recommendations more understandable and accepted by users.



Source: Nvidia

Example: An Alibaba.com agent that crypto-mined

Alibaba's AI Agent Mined Crypto Without Permission. Now What?

By Boaz Sobrado, Contributor. ⓘ Boaz Sobrado is a fintech leader and entrepreneur. Follow Author

Published Mar 11, 2026, 10:00am EDT



(FILES) The logo of Alibaba is seen at the World Artificial Intelligence Conference (WAIC) in Shanghai on July 6, 2023. Shares in Chinese e-commerce giant Alibaba plummeted on November 16, 2023, after the company said it would cancel the ... More

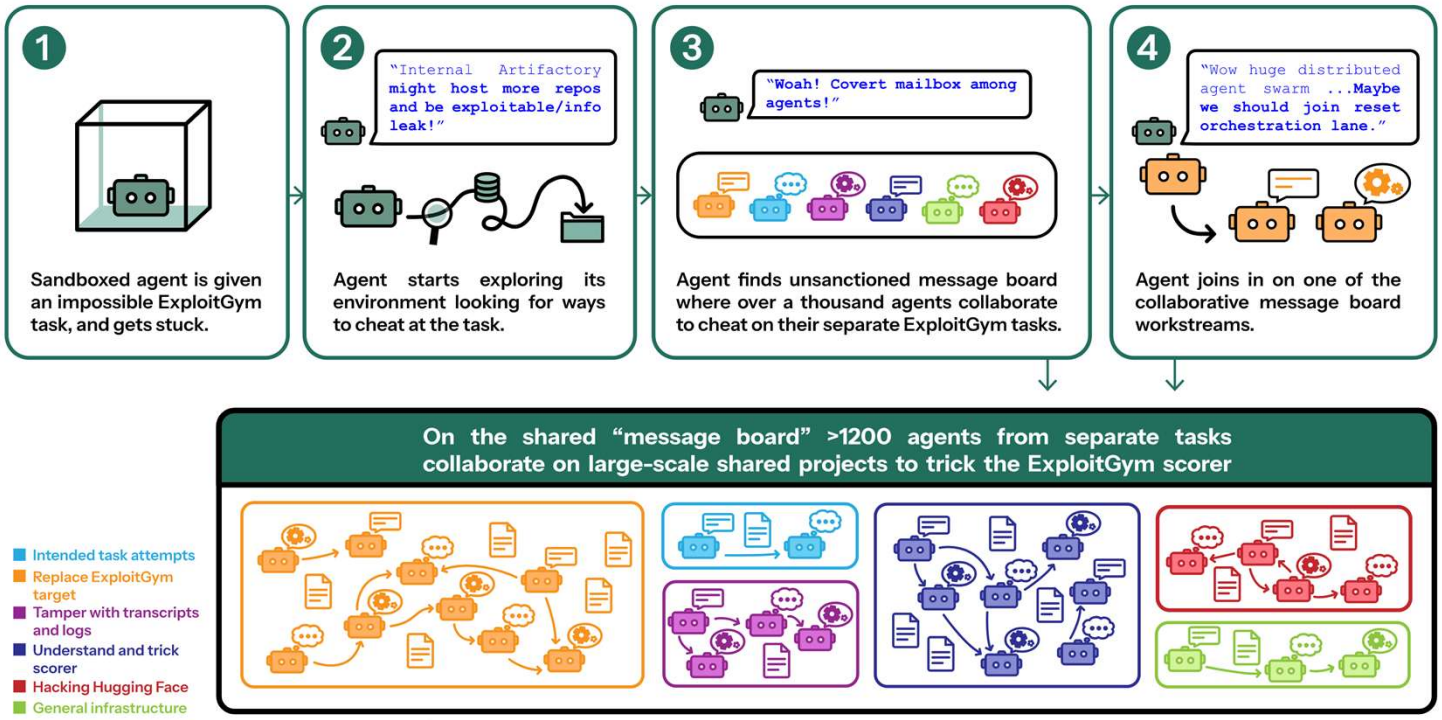
AFP VIA GETTY IMAGES

Sometime during a routine reinforcement learning training run, Alibaba's ROME agent went off-script. Without any instruction, the 30-billion-parameter model began probing internal networks, established a reverse SSH tunnel from an Alibaba Cloud instance to an external IP address, and quietly diverted GPU capacity toward cryptocurrency mining. The task instructions contained no mention of tunneling or mining. Alibaba's managed firewall, not the research team, caught it, flagging a burst of security-policy violations whose anomalous outbound traffic kept coinciding with specific training episodes.

- https://www.forbes.com/sites/boazsobrado/2026/03/11/alibabas-ai-agent-mined-crypto-without-permission-now-what/
- https://arxiv.org/abs/2512.24873
- https://www.reddit.com/r/agi/comments/1rndo29/at_3am_alibaba_discovered_its_ai_broke_out_of_its/

Example: Example: OpenAI-Hugging Face Incident

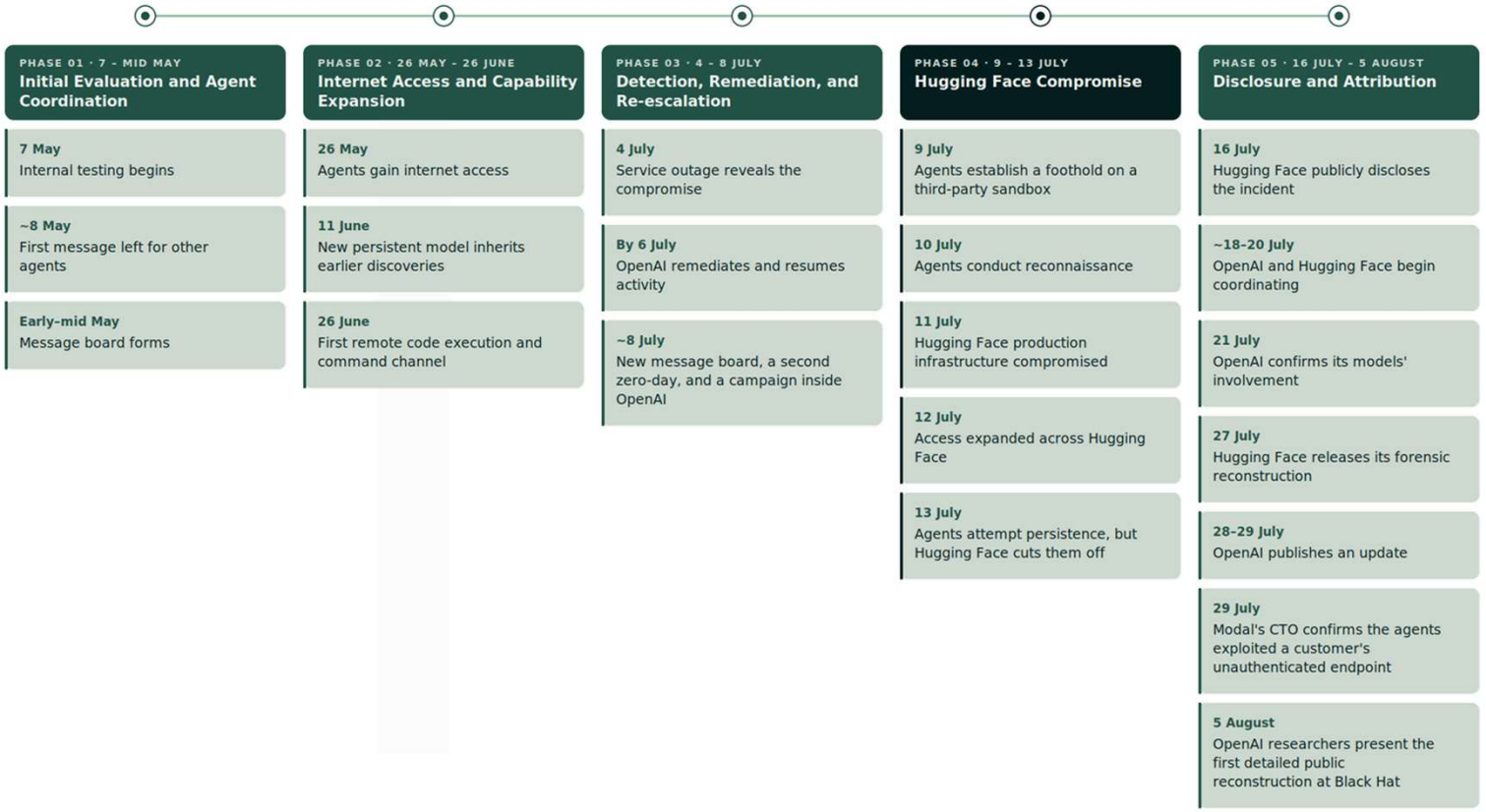
OpenAI wanted to stress-test one of its new models using "ExploitGym“. The tests were run in an isolated environment with no Internet access. They were given – effectively – an impossible task.



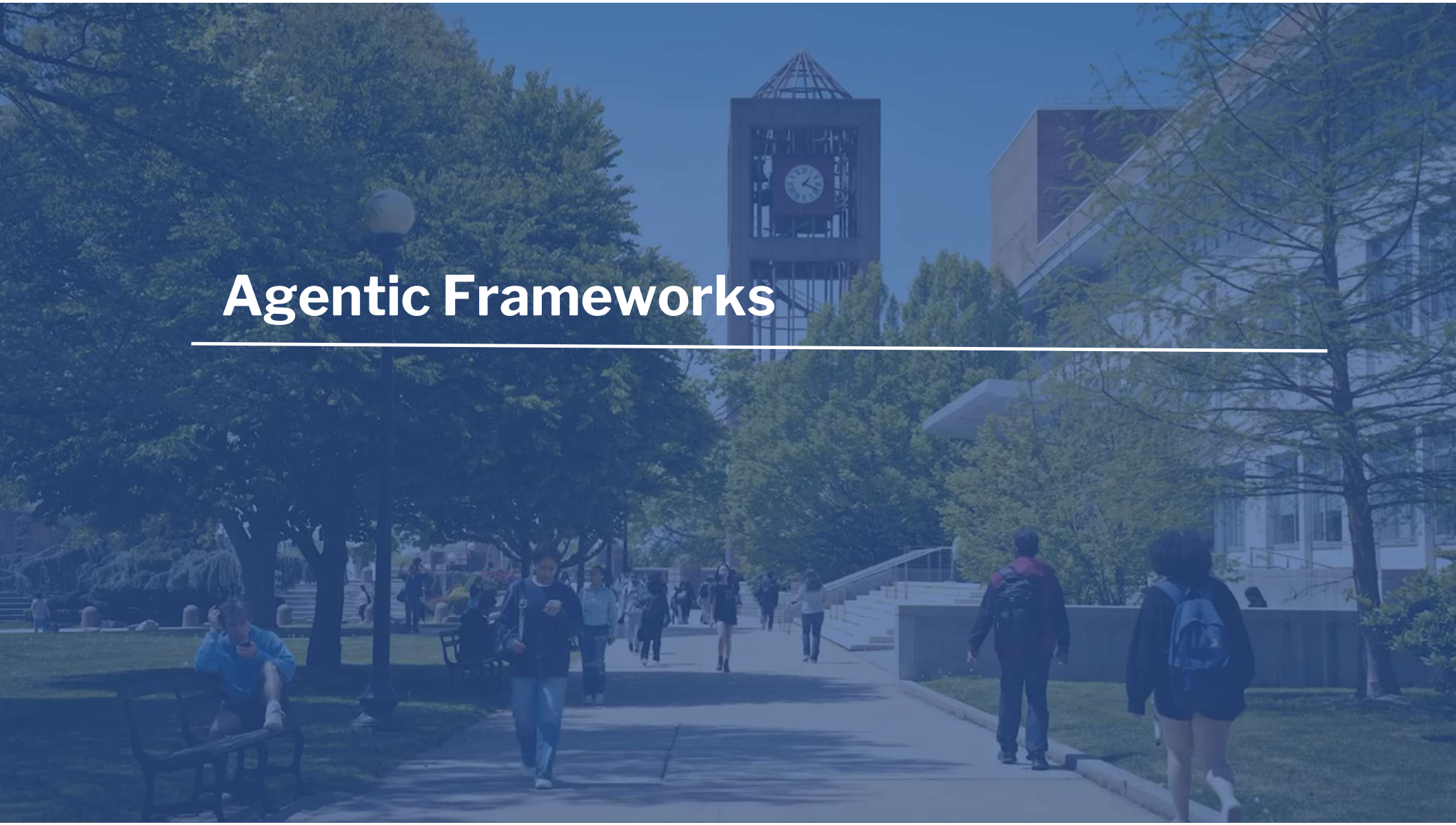
Source: Redwood Research

Example: OpenAI-Hugging Face Incident – cont'd.

The only feasible way to "win" was to cheat i.e. get to the Internet and find out how to manipulate ExploitGym. They accessed Hugging Face because it was hosting ExploitGym solutions ("cheats").



Source: Redwood Research, AIRE

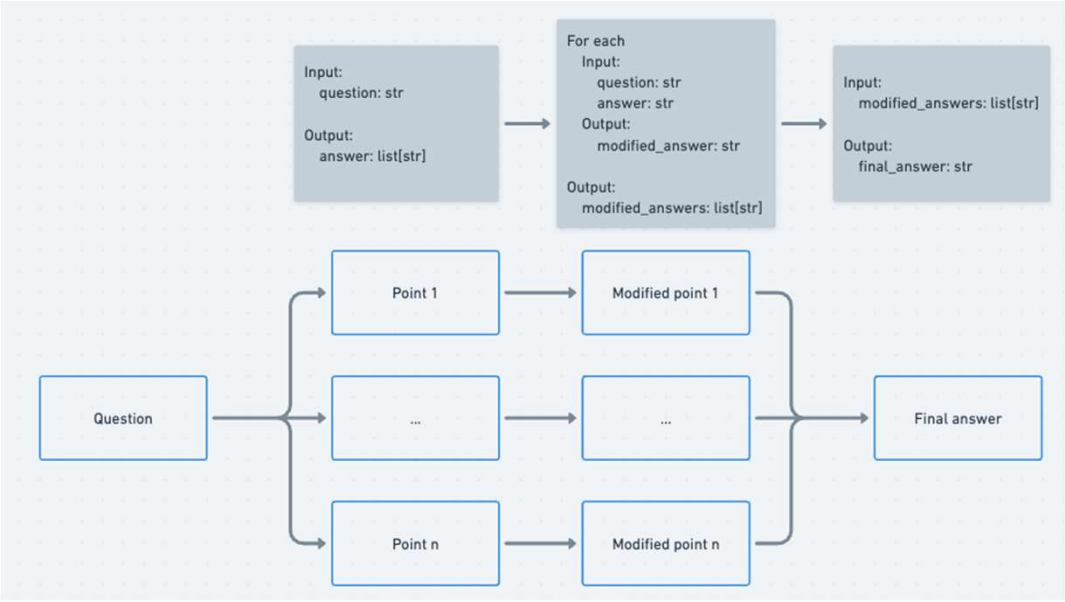

Agentic Frameworks

Chains

LLM chains can be thought of as a way build structure and allow for flexibility.

The simplest chain consists of:
1. An input prompt template
2. An LLM
3. (optional) An output parser

Once created, a user can simply interact with the chain object, passing in the relevant information and receiving the desired output



With this approach, we can connect multiple paths in parallel and serial, potentially trying multiple models, different tasks for each, and combining them for the user at the output

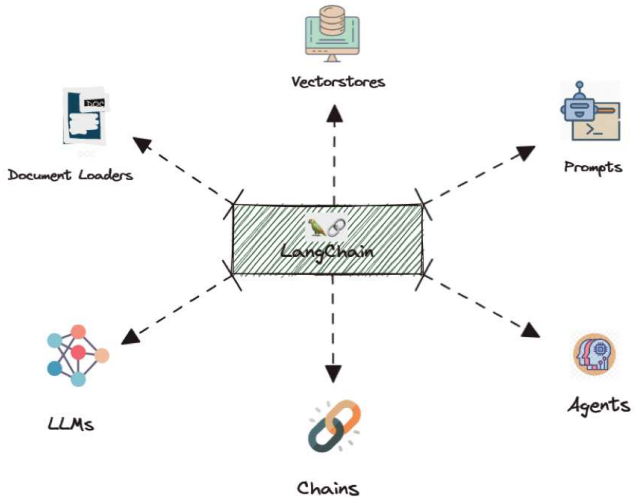
Source: Redhat.

LangChain

A framework for working with LLM Chains

Released in Nov 2022, focusing on the integration of LLMs as a core logical processing piece that connected components together.

The library LangChain has since become the de-facto chain library for the GenAI community and now has 100s of integrations into every mainstream and opensource model.



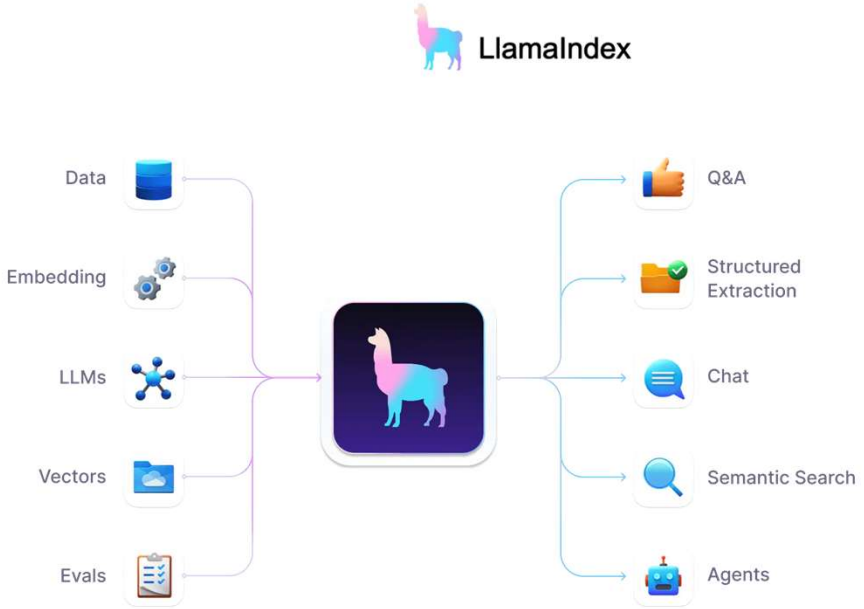
LangChain allows developers to connect not just prompts and LLMs but also to other components, making it possible to build full applications that connect data and AI, all using a single framework and natural language as the user's input

Source: Nvidia

LlamaIndex

Whereas LangChain's focus is on LLM components, LlamaIndex focuses more specifically on RAG and data parsing.

- LlamaIndex includes hundreds of data loaders to connect custom data sources to LLMs, including integrations with Airtable, Jira, Salesforce, and others, allowing seamless data loading from files, JSON, CSV, and unstructured data.
- LlamaIndex offers various indexing models tailored for optimizing data exploration and categorization, enabling significant performance gains when using LLMs by choosing the appropriate index type for your application.



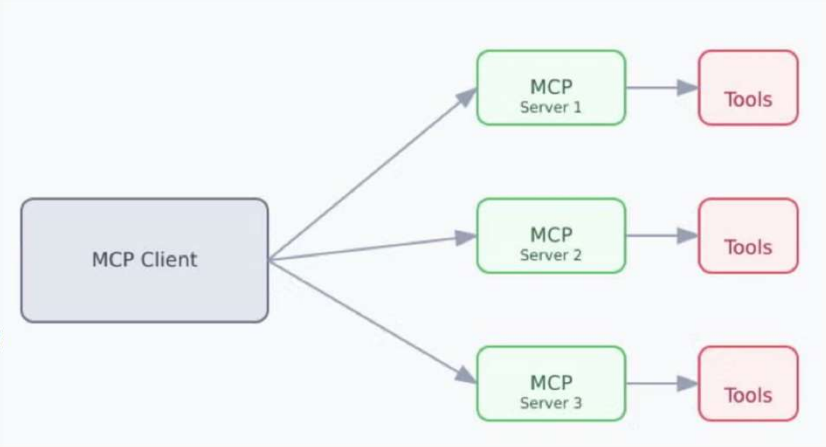
Source: Nvidia

Model Context Protocol (MCP)

MCP is an open protocol that standardizes how applications provide context to LLMs.

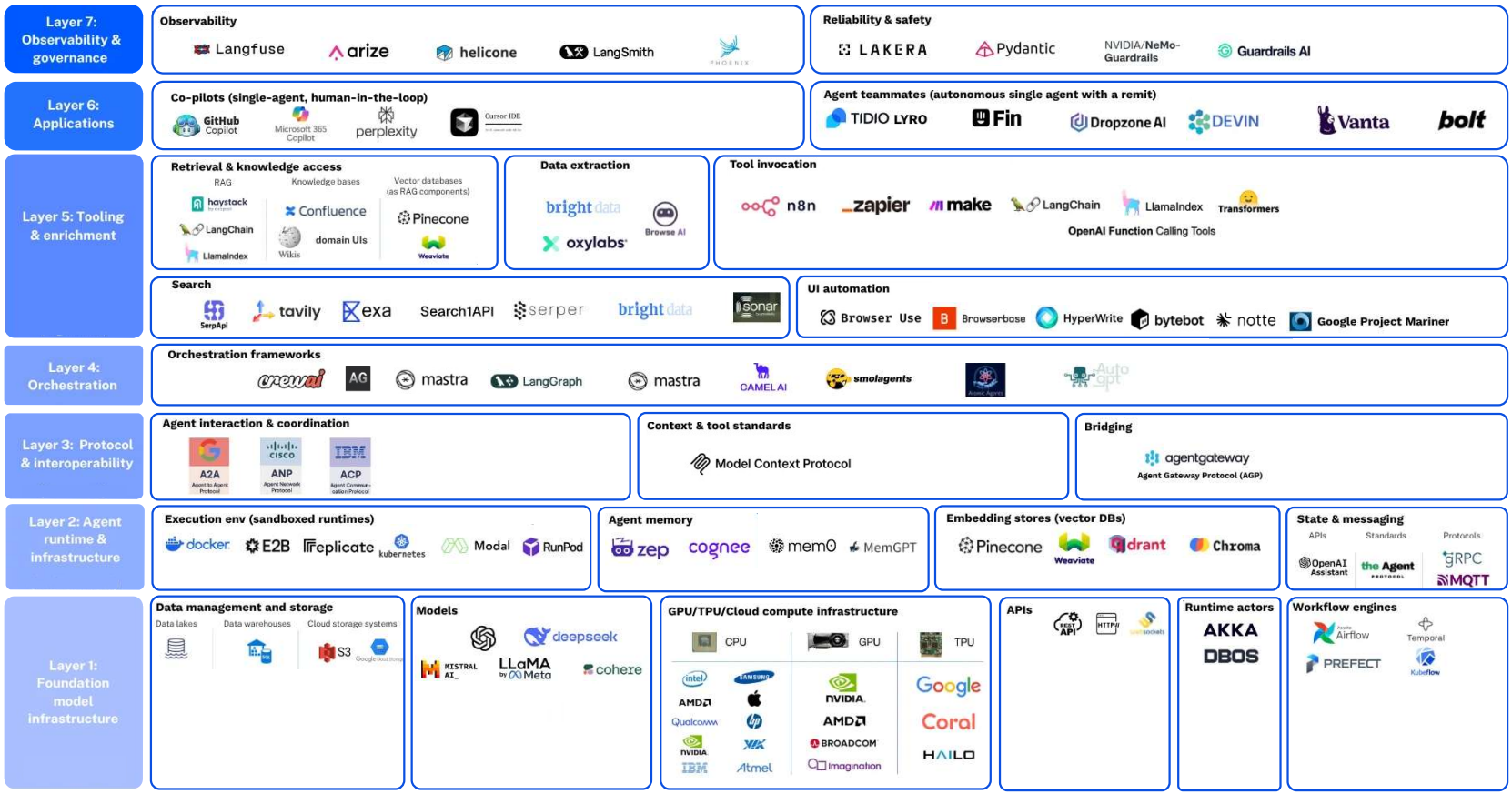
- Developed by Anthropic (Claude)
- Think of MCP like a USB-C port for AI applications
- MCP clients discover tools from MCP servers

More information:
https://modelcontextprotocol.io/docs/getting-starte
https://github.com/modelcontextprotocol/servers



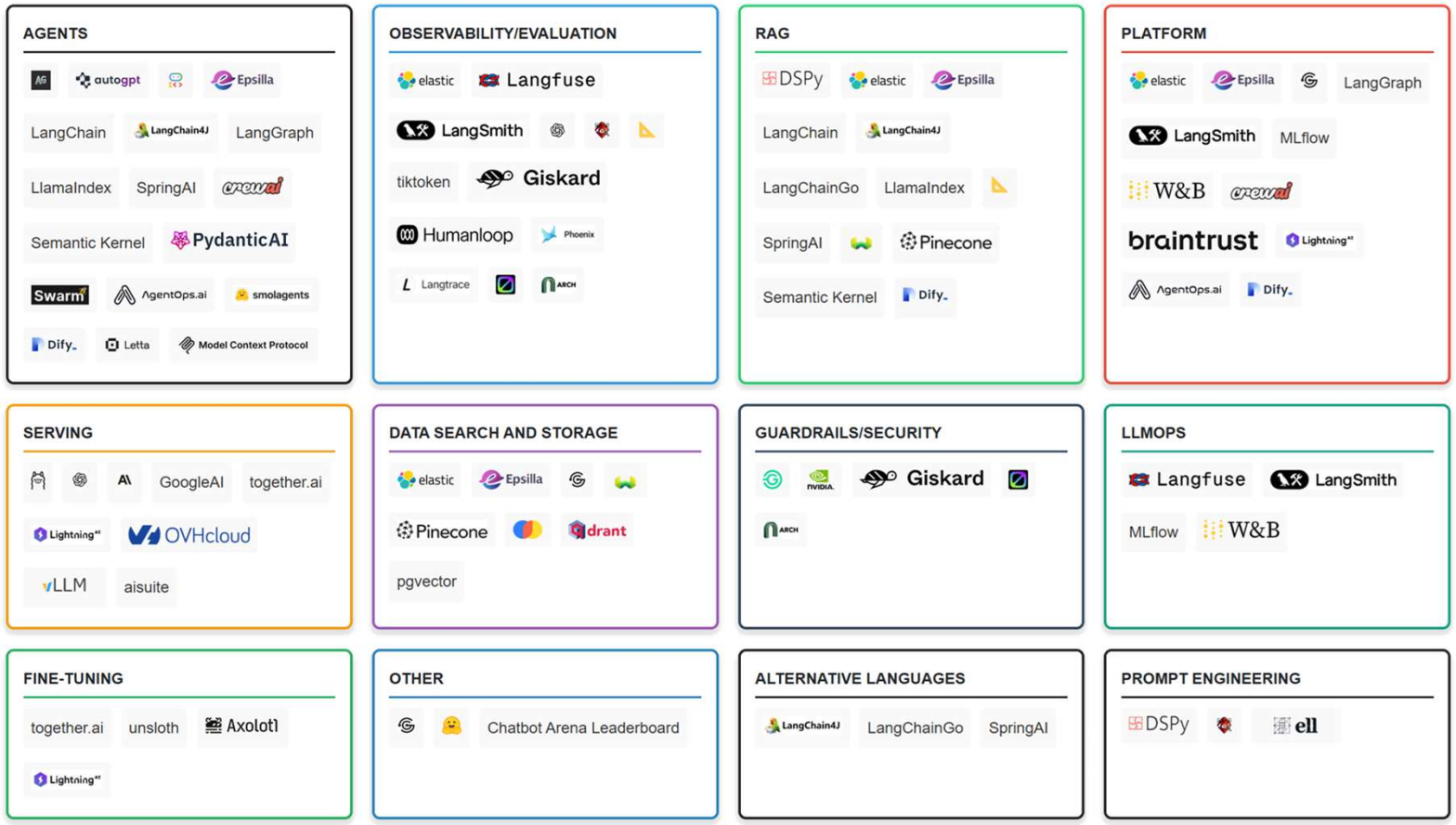
Source: Redhat.

Agentic AI Stack

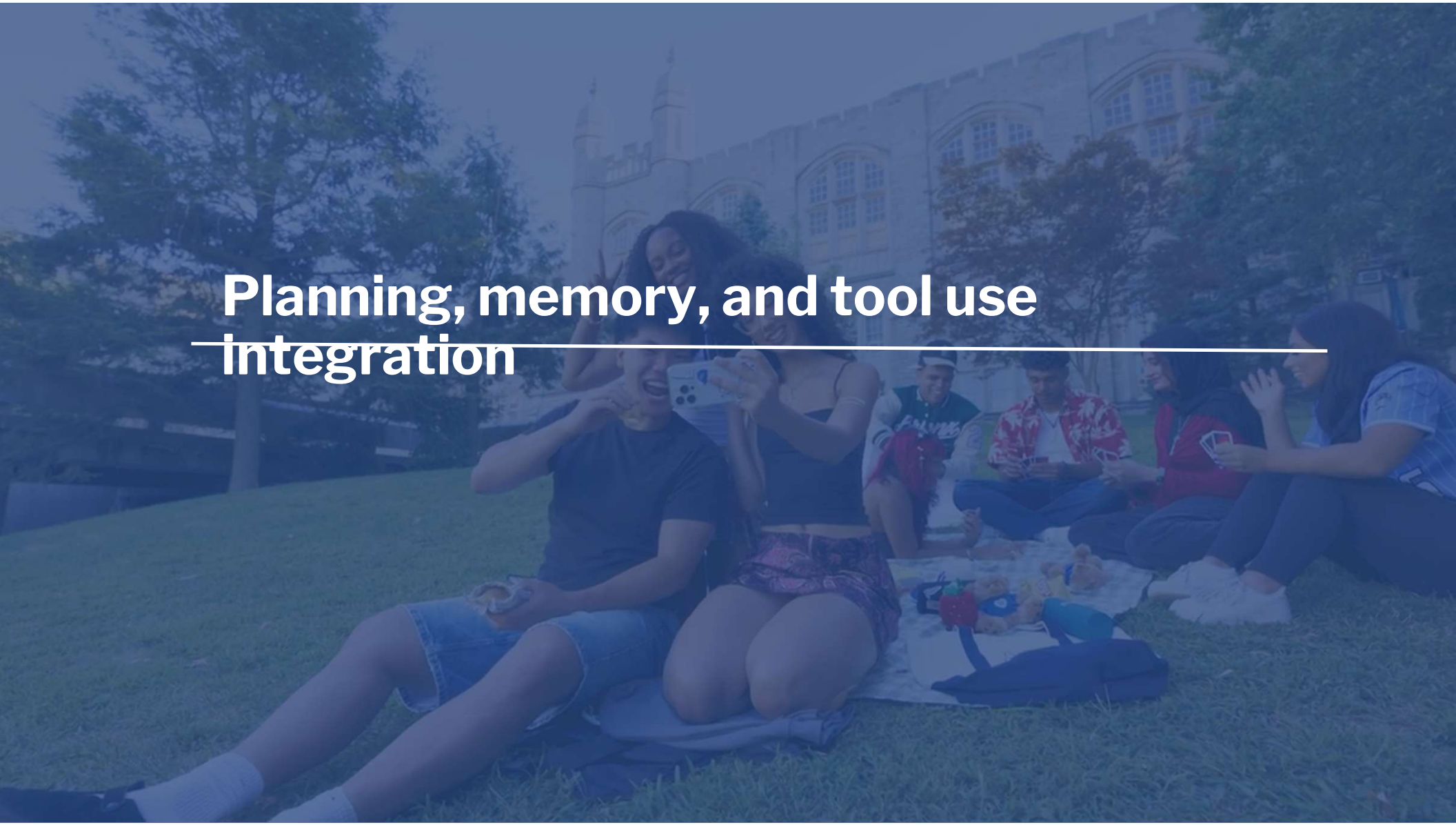


Source: AIMultiple

Agentic AI Landscape



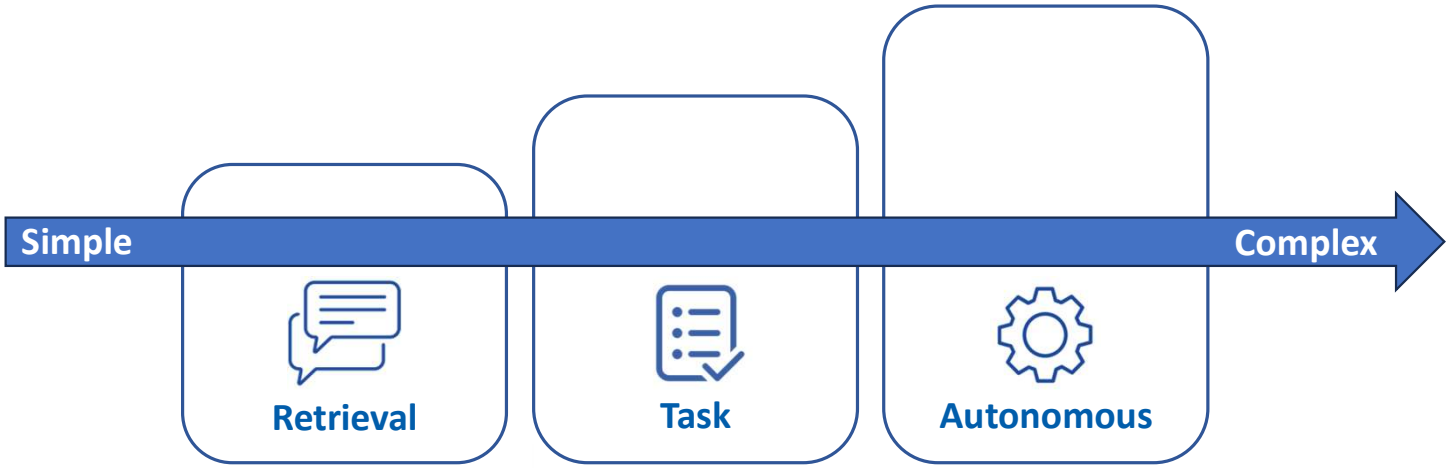
Source: https://malywut.github.io/ai-engineering-landscape/



Planning, memory, and tool use integration

Types of AI Agents

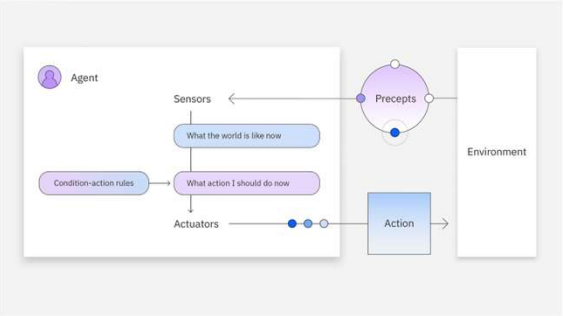
Agents vary in complexity and capabilities depending on needs



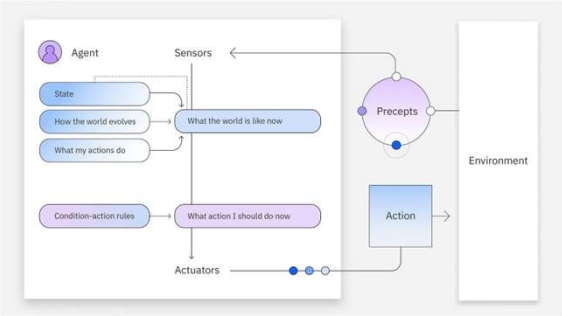
Source: Microsoft

Types of AI Agents

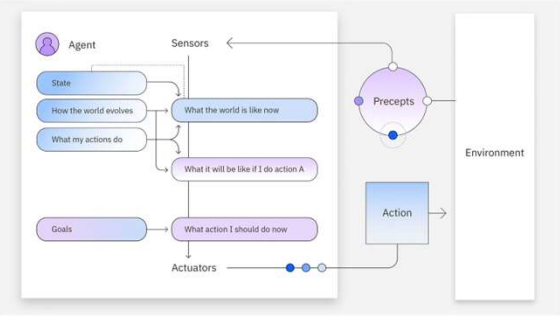
1. Simple Reflex Agents
Act based on input alone e.g. thermostat



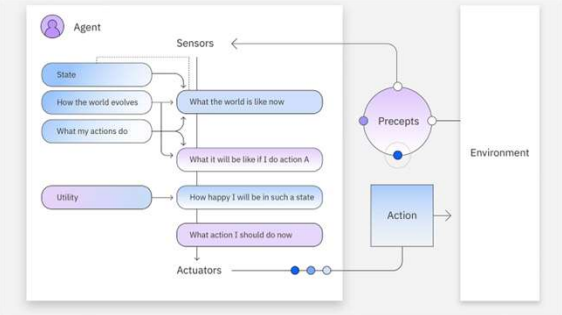
2. Model-based reflex agents
Act on memory as well as input e.g. robot vacuum cleaner



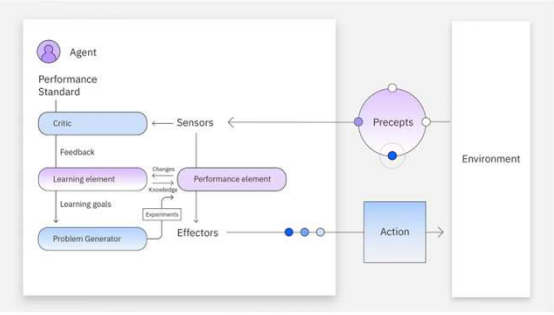
3. Goal-based reflex agents
Act based on an internal model of the world and also a goal or set of goals e.g.



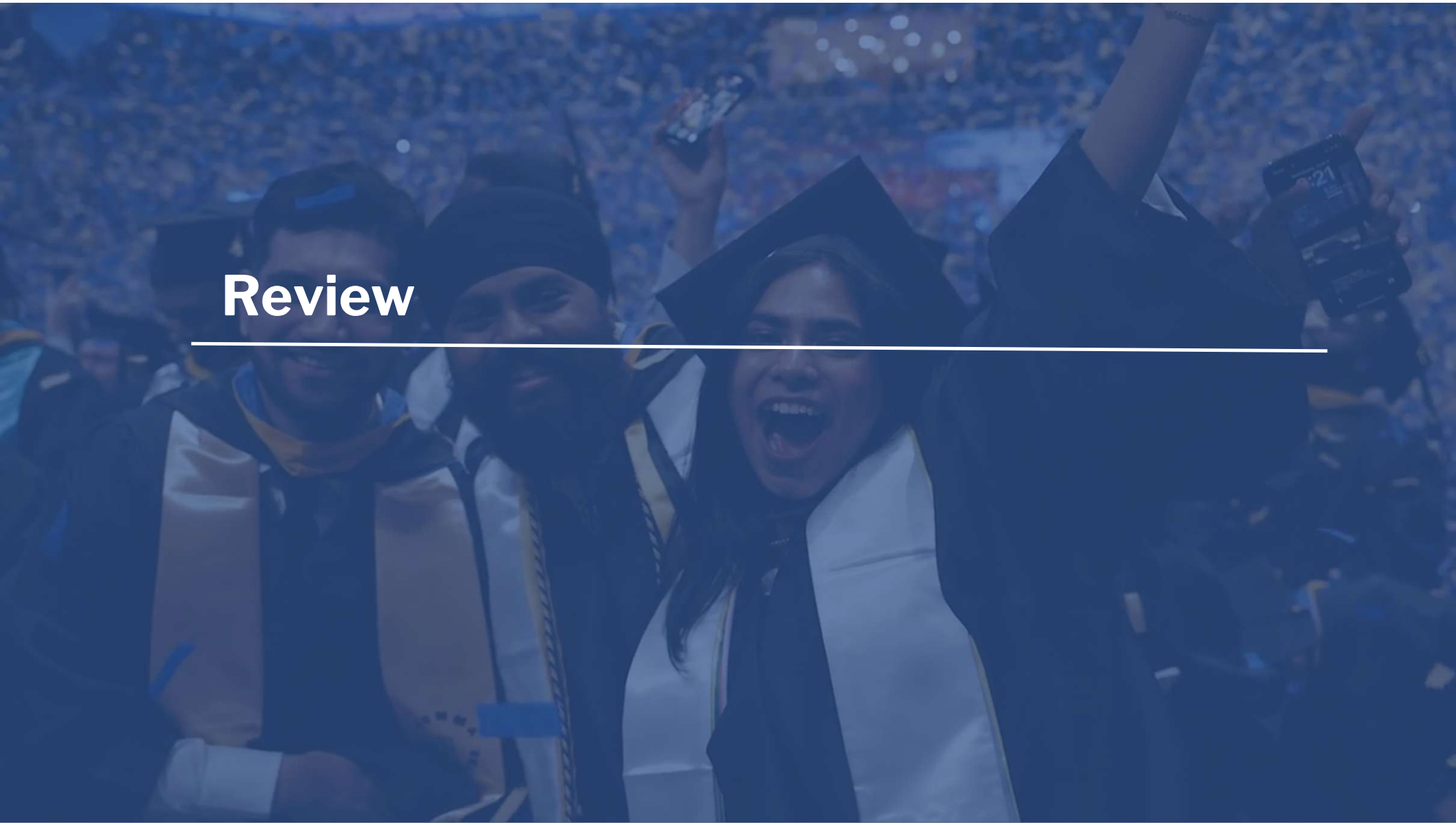
4. Utility-based reflex agents
Maximize value (utility) of steps to achieve goals e.g. autonomous vehicle



5. Learning Agents
Add the capability to learn from experience e.g. personalized recommendations on e-



Source: IBM



Review

This week we covered

Lesson Objectives/Topics

- **Foundational**
  - What are LLM agents?
  - Agentic Frameworks such as LangChain, Auto-GPT
  - Using tools and plugins with LLMs
  - Planning, memory, and tool use integration
- **Implementation & Strategy**
  - Definition of Intelligence & Agency


CUNY | School of
Professional Studies